

CS 201: Game Development & Physics Mechanics Pair-Programming Chat History & LLM Interaction Report

Course Project: Lava Escape Repair Lab

Student Name: [Your Name]

Student ID: [Your Student ID]

Date: October 6, 2026

Repository: 52_lava-escape

1. Context & Task Overview

This report documents the pair-programming and debugging workflow conducted with an LLM assistant for the Lava Escape Repair Lab. The project is a vertical platformer survival game built with Pygame.

The lab required:

1. Identifying and resolving a startup camera visibility issue.
2. Complete analysis and implementation of Task 1: Platform Underside Collision Snapping Bug.

3. Progressive implementation of optional mechanics:

- * Task 2: Crumbling Platform Hazards

- * Task 3: High-Velocity Spring Platforms

- * Task 4: Rising Danger HUD & Lava Burst Surges

2. Complete Chat History Log

--- Session Phase 1: Environment & Startup Bug Diagnosis ---

User:

I am working on the Lava Escape Repair Lab. I ran the original game before making any changes. The game window opens and the platforms/HUD are visible, but the player character is not visible, so I cannot actually play the game. Please inspect the existing code carefully and diagnose why the player is not visible.

Requirements:

- * Do NOT implement Tasks 1, 2, 3, or 4 yet.
- * First explain exactly where the player is spawned, where the ground is, and how the camera is initialized.
- * Explain why this causes the player to be outside the visible 500x640 game window.
- * Propose the smallest fix that makes the player visible while preserving intended behavior.

Assistant Response:

Diagnosis:

1. Initial Coordinates:

* Ground Level: Defined in game/game_engine.py as $\text{GROUND_Y} = \text{HEIGHT} + 200 = 840$.

* Player Spawn: In `GameEngine.reset()`, ground is placed at $y = 840$, and player spawns at $y = 840 - 50 = 790$.

* Camera Initialization: In `reset()`, `self.cam_y` is initialized directly to 0.

2. Root Cause:

Player screen position is computed via `self.rect.move(0, -int(self.cam_y))`. With `cam_y = 0` and player world position `y = 790`, the rendered y-coordinate is `790 - 0 = 790`. Because the window height is 640, the player spawns 150 pixels below the bottom viewport edge.

Proposed Minimal Fix:

Initialize `self.cam_y` inside `GameEngine.reset()` to center on the player's initial position:
`self.cam_y = self.player.rect.centery - HEIGHT // 2`

--- Session Phase 2: Task 1 Bug Analysis & Isolation ---

User:

I am completing Task 1 of my Lava Escape Repair Lab. Please inspect the existing codebase, especially the player/platform collision and physics logic, and identify the cause of the "Platform Underside Collision Snapping Bug".

Task 1 Requirement:

"When jumping up through platforms, the player's feet prematurely snap to the top surface from below or clip through solid edges rather than passing cleanly through one-way ledges. Ensure landing resolution only triggers when the player is descending and strictly clears the platform's upper edge."

Explain which file and code section causes the bug, why it snaps when moving upward, and the minimal fix proposed.

Assistant Response:

1. Affected File and Code Section:

* File: `game/player.py`

* Section: The collision loop inside `Player.update()`:

`if self.rect.colliderect(p) and self.vel_y > 0 and self.rect.bottom <= p.bottom + 10:`

2. Cause of the Bug:

The check `self.rect.bottom <= p.bottom + 10` compares the player's feet to the bottom edge of the platform rather than its top edge. When jumping upward through a platform, the player's bounding box intersects the platform bottom. If vertical velocity transitions or

evaluates positive near peak jump height while inside this 10px threshold, the engine prematurely snaps `self.rect.bottom = p.top`.

3. Minimal Proposed Fix:

Track `prev_bottom` before applying vertical movement, and strictly enforce landing criteria:

```
prev_bottom = self.rect.bottom
self.rect.y += int(self.vel_y)
```

for `p` in `platforms`:

```
if self.vel_y > 0 and prev_bottom <= p.top and self.rect.bottom >= p.top:
```

```
if self.rect.right > p.left and self.rect.left < p.right:
```

```
self.rect.bottom = p.top
```

```
self.vel_y = 0
```

```
self.on_ground = True
```

--- Session Phase 3: Task 2 - Crumbling Platform Hazards ---

User:

Implement Task 2: Crumbling Platform Hazards.

Assistant Response:

Changes Summary:

1. Created a `Platform` class in `game/world.py` extending `pygame.Rect` with attributes: `p.type`, `is_crumbling`, `timer`, and `destroyed`.

2. Added visual horizontal shaking logic (`random.randint(-3, 3)`) and disintegration after 45 frames (~0.75 seconds).

3. Updated `Player.update()` in `game/player.py` to trigger `p.trigger_crumble()` upon landing.

4. Updated `GameEngine.update()` in `game/game_engine.py` to update platform timers and filter out destroyed platforms.

--- Session Phase 4: Task 3 - High-Velocity Spring Platforms ---

User:

Implement Task 3: High-Velocity Spring Platforms.

Assistant Response:

Changes Summary:

1. Expanded Platform types in game/world.py to include 'spring' with color `SPRING_COLOR = (60, 220, 100)`.
2. Modified platform generation in `generate_platforms()` to assign a 20% spawn rate for spring platforms.
3. Updated collision resolution in game/player.py: when stepping on a spring platform, `self.vel_y` is set to -21 (launch force) and `self.on_ground` remains False.

--- Session Phase 5: Task 4 - Rising Danger HUD & Lava Burst Surges ---

User:

Implement Task 4: Rising Danger HUD & Lava Burst Surges.

Assistant Response:

Changes Summary:

1. Added surge timers to GameEngine:
 - * `surge_interval = 300` (~5 seconds)
 - * `surge_duration = 120` (~2 seconds)
2. During surge phases, lava ascent speed multiplies by 2.5x.

3. Added `_draw_danger_hud()` to render a dynamic danger gauge bar scaling with current lava speed, alongside a blinking "!! LAVA SURGE !!" banner overlay.

3. Code Modifications Record

1. `game/game_engine.py`:

* Primary Changes: Fixed startup camera initialization (`cam_y`), integrated platform lifecycle management, and added Danger HUD/Surge rendering.

2. `game/player.py`:

* Primary Changes: Corrected one-way platform landing check (`prev_bottom <= p.top`), integrated crumbling trigger and spring bounce response.

3. `game/world.py`:

* Primary Changes: Implemented Platform subclassing `pygame.Rect` to handle shaking, disintegration timers, type attributes, and distinct color codes.

4. Verification & Submission Criteria Checklist

[X] Startup Bug Resolution: Verified player spawns in visible window bounds (y approx 320px).

[X] Task 1 Fix Verified: Confirmed player passes cleanly upward through platform undersides and lands only when descending.

[X] Task 2 Verified: Crumbling platforms shake upon landing and disintegrate within 0.75s.

[X] Task 3 Verified: Spring platforms trigger bonus upward jump launch (`vel_y = -21`).

[X] Task 4 Verified: Danger gauge correctly reflects lava speed and surge warnings appear.

[X] Pre-Fix / Post-Fix Gameplay Clips: Recorded 10-second before/after videos as required.

